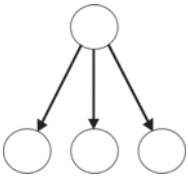
Chapter 4
Bayesian Classifiers



4.1 Introduction

Classification consists in assigning classes or labels to objects. There are two basic types of classification problems:

Unsupervised: in this case the classes are unknown, so the problem consists in dividing a set of objects into n groups or clusters, so that a class is assigned to each different group. It is also known as *clustering*.

Supervised: the possible classes or labels are known a priori, and the problem consists in finding a function or rule that assigns one or more classes to each object.

In both cases the objects are usually described by a set of properties or attributes. In this chapter we will focus on supervised classification.

From a probabilistic perspective, the problem of supervised classification consists in assigning to a particular object described by its attributes, $A_1, A_2, \ldots, A_n$, one of m classes, $C = \{c_1, c_2, \ldots, c_m\}$, such that the probability of the class given the attributes is maximized; that is:

$$Arg_C[Max\, P(C \mid A_1, A_2, \ldots, A_n)] \tag{4.1}$$

L. E. Sucar, *Probabilistic Graphical Models*, Advances in Computer Vision and Pattern Recognition, https://doi.org/10.1007/978-3-030-61943-5_4

If we denote the set of attributes as $\mathbf{A} = \{A_1, A_2, \ldots, A_n\}$, Eq. 4.1 can be written as: $Arg_C[Max\, P(C \mid \mathbf{A})]$.

There are many ways to build a classifier, including decision trees, rules, neural networks and support vector machines, among others.[1] In this book we will cover classification models based on probabilistic graphical models, in particular Bayesian classifiers. Before we describe the Bayesian classifier, we will present a brief introduction to how classifiers are evaluated.

4.1.1 Classifier Evaluation

To compare different classification techniques, we can evaluate a classifier in terms of several aspects. The aspects to evaluate, and the importance given to each aspect, depends on the final application of the classifiers. The main aspects to consider are:

Accuracy: it refers to how well a classifier predicts the correct class for unseen examples (that is, those not considered for learning the classifier).

Classification time: how long the classification process takes to predict the class, once the classifier has been trained.

Training time: how much time is required to learn the classifier from data.

Memory requirements: how much space in terms of memory is required to store the classifier parameters.

Clarity: if the classifier is easily understood by a person.

Usually the most important aspect is accuracy. This can be measured by predicting N unseen data samples, and determining the percentage of correct predictions. Thus, the accuracy in percentage is:

$$Acc = (N_C/N) \times 100 \tag{4.2}$$

Where N_C is the number of correct predictions.

A more detailed way to represent the performance of a classifier is by a *confusion matrix*. Each row of the matrix represents the instances for the predicted class while each column represents the instances for the actual class. In the case of binary classification (with classes positive and negative), there are four cells in the confusion matrix which represent:

- True positive (TP): number of instances of the positive class predicted as positive.
- True negative (TN): number of instances of the negative class predicted as negative.
- False positive (FP): number of instances of the negative class predicted as positive (Type I error).
- False negative (FN): number of instances of the positive class predicted as negative (Type II error).

[1] For an introduction and comparison of different types of classifiers we refer the interested reader to [11].

Table 4.1 Confusion matrix for binary classification

		Actual Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

The confusion matrix for binary classification is depicted in Table 4.1. This can be extended for N classes, in that case it will be an $N \times N$ matrix.

According to the previous notation, the accuracy for a binary classifier is:

$$Acc = \frac{TP + TN}{P + N} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.3}$$

Where P is the number of positive cases and N the number of negative cases.

When comparing classifiers, in general we want to maximize the classification accuracy; however, this is only *optimal* if the cost of a wrong classification is the same for all the classes. Consider a classifier used for predicting breast cancer. If the classifier predicts that a person has cancer but this is not true (false positive), this might produce an unnecessary treatment. On the other hand, if the classifier predicts no cancer and the person does have breast cancer (false negative), this might cause a delay in the treatment and even death. Clearly the consequences of the several types of errors are different.

When there is imbalance in the costs of misclassification, we must then minimize the expected cost (EC). For two classes, this is given by:

$$EC = FN \times P(-)C(- \mid +) + FP \times P(+)C(+ \mid -) \tag{4.4}$$

Where: FN is the false negative rate, FP is the false positive rate, $P(+)$ is the probability of positive, $P(-)$ is the probability of negative, $C(- \mid +)$ is the cost of classifying a positive as negative, and $C(+ \mid -)$ is the cost of classifying a negative as positive. Determining these costs may be difficult for some applications.

4.2 Bayesian Classifier

The formulation of the *Bayesian Classifier* is based on the application of the Bayes rule to estimate the probability of each class given the attributes:

$$P(C \mid A_1, A_2, \ldots, A_n) = P(C)P(A_1, A_2, \ldots, A_n \mid C)/P(A_1, A_2, \ldots, A_n) \tag{4.5}$$

Which can be written more compactly as:

$$P(C \mid \mathbf{A}) = P(C)P(\mathbf{A} \mid C)/P(\mathbf{A}) \tag{4.6}$$

The classification problem, based on Eq. 4.6, can be formulated as:

$$Arg_C[Max[P(C \mid \mathbf{A}) = P(C)P(\mathbf{A} \mid C)/P(\mathbf{A})]] \tag{4.7}$$

Equivalently, we can express Eq. 4.7 in terms of any function that varies monotonically with respect to $P(C \mid \mathbf{A})$, for instance:

- $Arg_C[Max[P(C)P(\mathbf{A} \mid C)]]$
- $Arg_C[Max[log(P(C)P(\mathbf{A} \mid C))]]$
- $Arg_C[Max[(log P(C) + log P(\mathbf{A} \mid C)]]$

Note that the probability of the attributes, $P(\mathbf{A})$, does not vary with respect to the class, so it can be considered as a constant for the maximization.

Based on the previous equivalent formulations for solving a classification problem we will require an estimate of $P(C)$, known as the prior probability of the classes, and $P(\mathbf{A} \mid C)$, known as the *likelihood*; $P(C \mid \mathbf{A})$ is the posterior probability. Therefore, to obtain the posterior probability of each class, we just need to multiply its prior probability by the likelihood which depends on the values of the attributes.[2]

The direct application of the Bayes rule results in a computationally expensive problem, as it was mentioned in Chap. 1. This is because the number of parameters in the likelihood term, $P(A_1, A_2, \ldots, A_n \mid C)$, increases exponentially with the number of attributes. This will not only imply a huge amount of memory to store all the parameters, but it will also be very difficult to estimate all the probabilities from the data or with the aid of a domain expert. Thus, the Bayesian classifier can only be of practical use for relatively *small* problems in terms of the number of attributes. An alternative is to consider some independence properties as in graphical models, in particular that all attributes are independent given the class, resulting in the *Naive Bayesian Classifier*.

4.2.1 Naive Bayesian Classifier

The naive or simple Bayesian classifier (NBC) is based on the assumption that all the attributes are independent given the class variable; that is, each attribute A_i is conditionally independent of all the other attributes given the class: $P(A_i \mid A_j, C) = P(A_i \mid C), \forall j \neq i$. Under this assumption, Eq. 4.5 can be written as:

$$P(C \mid A_1, A_2, \ldots, A_n) = P(C)P(A_1 \mid C)P(A_2 \mid C) \ldots P(A_n \mid C)/P(\mathbf{A}) \tag{4.8}$$

where $P(\mathbf{A})$ can be considered, as mentioned before, a normalization constant.

[2]The posterior probabilities of the classes will be affected by a constant as we are not considering the denominator in Eq. 4.7, that is, they will not add to one; however, they can be easily *normalized* by dividing each one by the sum of the posterior probabilities for all the classes.

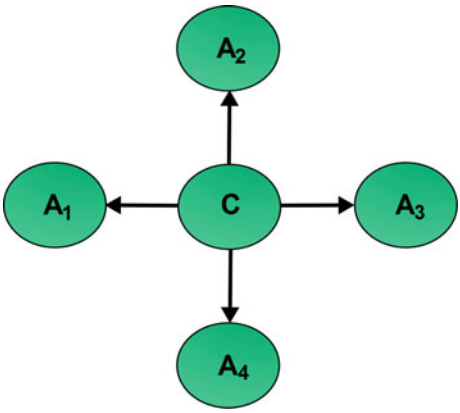


Fig. 4.1 An example of a naive Bayesian classifier with the class variable, C, and four attributes, $A_1, \ldots, A_4$

The naive Bayes formulation drastically reduces the complexity of the Bayesian classifier, as in this case we only require the prior probability (one dimensional vector) of the class, and the n conditional probabilities of each attribute given the class (two dimensional matrices) as the parameters for the model. That is, the space requirement is reduced from exponential to linear in the number of attributes. Also, the calculation of the posterior is greatly simplified, as to estimate it (unnormalized) only n multiplications are required.

A graphical representation of the naive Bayesian classifier is shown in Fig. 4.1. This star like structure depicts the property of conditional independence between all the attributes given the class—as there are not arcs between the attribute nodes.

Learning a NBC consists in estimating the prior probability of the class, $P(C)$, and the conditional probability table (CPT) of each attribute given the class, $P(A_i \mid C)$. These can be obtained via subjective estimates from an expert or from the data by maximum likelihood.[3]

The probabilities can be estimated from data using, for instance, maximum likelihood estimation. The prior probabilities of the class variable, C, are given by:

$$P(c_i) \sim N_i/N \tag{4.9}$$

Where N_i is the number of times c_i occurs in the N samples.

The conditional probabilities of each attribute, A_j can be estimated as:

$$P(A_{jk} \mid c_i) \sim N_{jki}/N_i \tag{4.10}$$

Where N_{jki} is the number of times that the attribute A_j takes the value k and it is from class i, and N_i is the number of samples of class c_i in the data set.

[3] We will cover parameter estimation in detail in the chapter on Bayesian Networks.

Table 4.2 Data set for the golf example

Outlook	Temperature	Humidity	Windy	Play
Sunny	High	High	False	No
Sunny	High	High	True	No
Overcast	High	High	False	Yes
Rain	Medium	High	False	Yes
Rain	Low	Normal	False	Yes
Rain	Low	Normal	True	No
Overcast	Low	Normal	True	Yes
Sunny	Medium	High	False	No
Sunny	Low	Normal	False	Yes
Rain	Medium	Normal	False	Yes
Sunny	Medium	Normal	True	Yes
Overcast	Medium	High	True	Yes
Overcast	High	Normal	False	Yes
Rain	Medium	High	True	No

Once the parameters have been estimated, the posterior probability can be obtained just by multiplying the prior by the likelihood for each attribute. Thus, given the values for m attributes, $a_1, \ldots a_m$, for each class c_i, the posterior is proportional to:

$$P(c_i \mid a_1, \ldots a_m) \sim P(c_i)P(a_1 \mid c_i) \cdots P(a_m \mid c_i) \tag{4.11}$$

The class c_k that maximizes the previous equation will be selected.[4]

Returning to the *golf* example, Table 4.2 depicts 14 records with 5 variables: four attributes (Outlook, Temperature, Humidity, Windy) and one class (Play). A NBC for the golf example is depicted in Fig. 4.2, including some of the required probability tables.

In summary, the main advantages of the naive Bayesian classifier are:

- The low number of required parameters, which reduces the memory requirements and facilitates learning them from data.
- The low computational cost for inference (estimating the posteriors) and learning.
- The relatively good performance (classification precision) in many domains.
- A simple and intuitive model.

While its main limitations are the following:

- In some domains, the performance is reduced given that the conditional independence assumption is not valid.

[4]This assumes that the misclassification cost is the same for all classes; if these costs are not the same, the class that minimizes the misclassification cost should be selected.

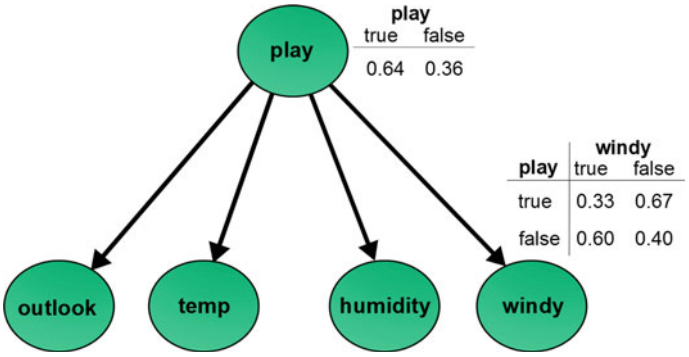


Fig. 4.2 A naive Bayesian classifier for the golf example. Two of the required probability tables are shown: $P(play)$ and $P(windy \mid play)$. The conditional probabilities of the other three attributes given the class will also be required to complete the parameters of this classifier

- If there are continuous attributes, these need to be discretized (or consider alternative models such as the Gaussian naive Bayes).

In the next section we introduce the Gaussian naive Bayesian classifier. Then we will cover other models that consider certain dependencies between attributes. Later we will describe techniques that can eliminate or join attributes, as another way to overcome the conditional independence assumption.

4.3 Gaussian Naive Bayes

In this case, the attributes are continuous random variables, and the class is discrete. Thus, each attribute is modeled by some continuous probability distribution over the range of its values. A common assumption is that within each class, the values of numeric attributes are normally distributed. That is:

$$P(A_i \mid C = c_j) = (1/\sqrt{2\pi\sigma})e^{-\frac{(a_i-\mu)^2}{2\sigma^2}} \tag{4.12}$$

Where μ and σ are the mean and standard deviation, respectively, of the distribution of attribute A_i given class c_j. So for each attribute given each class value, the Gaussian naive Bayes (GNB) requires to represent such a distribution in terms of its mean and standard deviation. Additionally, as in the naive Bayes, the prior probability of each class is required.

Learning a GNB consists in estimating the prior probabilities of the classes, and the mean and standard deviation of each attribute given the class. The prior of the class is obtained as for the naive Bayes via maximum likelihood estimation. The maximum likelihood estimates of the mean and standard deviation of each attribute

given the class; that is of a normal distribution, are the sample average and the sample standard deviation. These can be easily obtained from data.
Sample average:

$$\mu_X \sim \overline{X} = (1/N) \sum_{i=1}^{N} x_i \tag{4.13}$$

Sample standard deviation:

$$\sigma_X \sim S_X = \sqrt{\sum_{i=1}^{N} (x_i - \overline{X})/(N-1)} \tag{4.14}$$

In the inference stage, for estimating the posterior probabilities of each class given the values of the attributes, we do the same as for the naive Bayes, multiply the prior probability of the class by the probabilities of each attribute given the class:

$$P(c_i \mid a_1, \ldots a_m) \sim P(c_i) P(a_1 \mid c_i) \cdots P(a_m \mid c_i) \tag{4.15}$$

Where $P(a_j \mid c_i)$ is obtained by calculating it from the normal distribution equation according to the value of a_j and the corresponding parameters (mean and standard deviation).

Note that with this formulation we can easily mix discrete and continuous attributes, estimating the CPTs for the discrete attributes and the mean and standard deviation for the continues attributes (assuming Gaussian distributions); and then simply applying the naive Bayes equation for estimating the posterior probabilities of each class.

4.4 Alternative Models: TAN, BAN

The general Bayesian classifier and the naive Bayesian classifier are the two extremes of possible dependency structures for Bayesian classifiers; the former represents the most complex structure with no independence assumptions, while the latter is the simplest structure that assumes that all the attributes are independent given the class. Between these two extremes there is a wide variety of possible models of varying complexities. Two interesting alternatives are the TAN and BAN classifiers.

The *Tree augmented Bayesian Classifier*, or TAN, incorporates some dependencies between the attributes by building a directed tree among the attribute variables. That is, the n attributes form a graph which is restricted to a directed tree that represents the dependency relations between the attributes. Additionally there is an arc between the class variables and each attribute. The structure of a TAN classifier is depicted in Fig. 4.3.

If we take out the limitation of a tree structure between attributes, we obtain the *Bayesian Network augmented Bayesian Classifier*, or BAN, which considers that

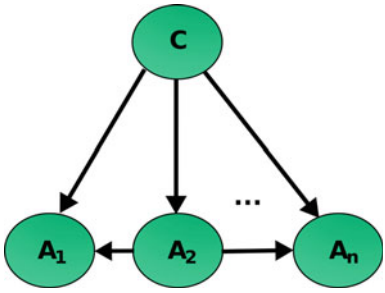


Fig. 4.3 An example of a TAN classifier

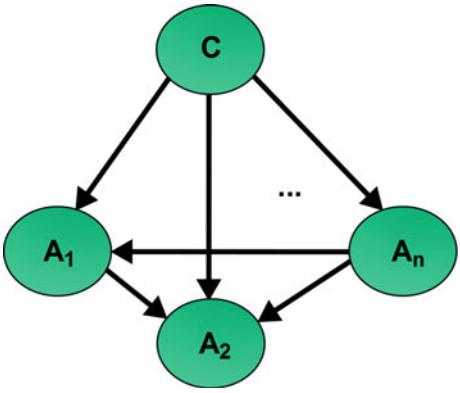


Fig. 4.4 An example of a BAN classifier

the dependency structure among the attributes constitutes a directed acyclic graph (DAG). As with the TAN classifier, there is a directed arc between the class node and each attribute. The structure of a BAN classifier is depicted in Fig. 4.4.

The posterior probability for the class variable given the attributes can be obtained in a similar way as with the NBC; however, now each attribute not only depends on the class but also on other attributes according to the structure of the graph. Thus, we need to consider the conditional probability of each attribute given the class and its *parent* attributes:

$$P(C \mid A_1, A_2, \ldots, A_n) = P(C)P(A_1 \mid C, Pa(A_1))P(A_2 \mid C, Pa(A_2)) \ldots P(A_n \mid C, Pa(A_n))/P(\mathbf{A}) \tag{4.16}$$

where $Pa(A_i)$ is the set of parent attributes of A_i according to the attribute dependency structure of the TAN or BAN classifier.

The TAN and BAN classifiers can be considered as particular cases of a more general model, that is, Bayesian networks, which will be covered in Chap. 7. In Chaps. 7 and 8, we will cover different techniques for inference and for learning Bayesian networks, which can be applied to obtain the posterior probabilities (inference) and the model (learning) for the TAN and BAN classifiers.

4.5 Semi-naive Bayesian Classifiers

Another alternative to deal with dependent attributes is to transform the basic structure of a naive Bayesian classifier, while maintaining a star or tree-structured network. This has as an advantage that the efficiency and simplicity of the NBC is maintained, and at the same time the performance is improved for cases where the attributes are not independent. These types of Bayesian classifiers are known as *Semi-Naive Bayesian Classifiers* (SNBC), and several authors have proposed different variants of SNBCs [12, 19].

The basic idea of the SNBC is to eliminate or *join* attributes which are not independent given the class, such that the performance of the classifier improves. This is analogous to *feature selection* in machine learning, and there are two types of approaches:

Filter: the attributes are selected according to a local measure, for instance the mutual information between the attribute and the class.
Wrapper: the attributes are selected based on a global measure, usually by comparing the performance of the classifier with and without the attribute.

Additionally, the learning algorithm can start from an empty structure and add (or combine) attributes; or from a full structure with all the attributes, and eliminate (or combine) attributes.

Figure 4.5 illustrates the two alternative operations to modify the structure of a NBC: (i) node elimination, and (ii) node combination, considering that we start from a full structure.

Node elimination consists in simply eliminating an attribute, A_i, from the model, this could be because it is not relevant for the class (A_i and C are independent); or because the attribute A_i and another attribute, A_j, are not independent given the class (which is a basic assumption of the NBC). The rationale for eliminating one of the dependent attributes is that if the attributes are not independent given the class, one of them is redundant and could be eliminated.

Node combination consists in merging two attributes, A_i and A_j, into a new attribute A_k, such that A_k has as possible values the cross product of the values of A_i and A_j (assuming discrete attributes). For example, if $A_i = a, b, c$ and $A_j = 1, 2$, then $A_k = a1, a2, b1, b2, c1, c2$. This is an alternative when two attributes are not independent given the class. By merging them into a single combined attribute, the independence condition is not longer relevant.

Thus, when two attributes are not independent given the class there are two alternatives: eliminate one or combine them into a single variable; in principle we should select the alternative that implies a higher improvement in the performance of the classifier, although in practice, this could be difficult to evaluate.

As mentioned before, there are several alternatives for learning a SNBC. A simple greedy scheme is outlined in Algorithm 4.1, where we start from a full NBC with all the attributes [10].

This process is repeated until there are no more superfluous or dependent attributes.

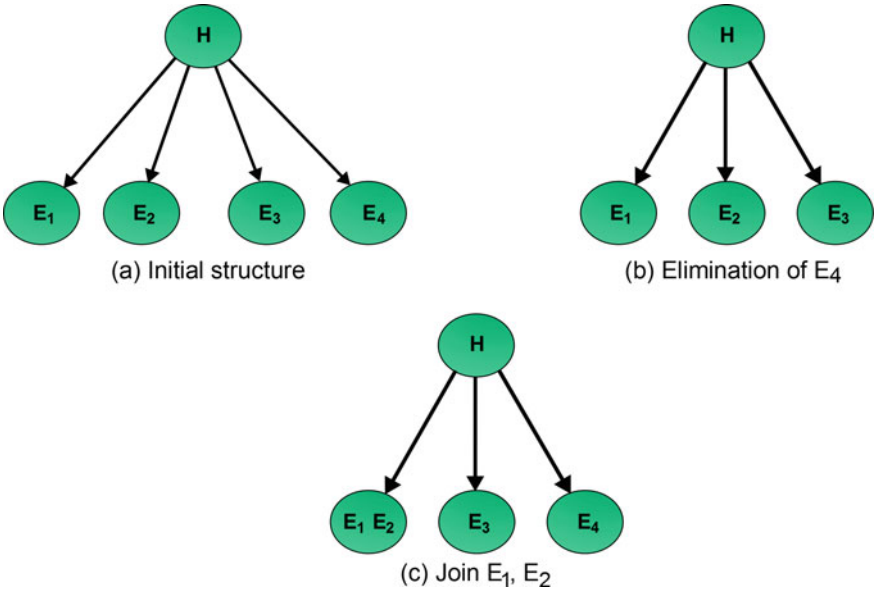


Fig. 4.5 Structural improvement: **a** initial structure, **b** one attribute is eliminated, **c** two attributes are joined into one variable

Algorithm 4.1 Structural Improvement Algorithm

Require: A, the attributes
Require: C, the class
/* The dependency between each attribute and the class is estimated (for instance using mutual information).*/
for all $a \in \mathbf{A}$ **do**
/* Those attributes below a threshold are eliminated.*/
if $MI(a, C) < \varepsilon$ **then**
Eliminate a
end if
end for
/* The remaining attributes are tested to see if they are independent given the class, for example, using conditional mutual information (CMI).*/
for all $a \in \mathbf{A}$ **do**
for all $b \in \mathbf{A} - a$ **do**
/* Those attributes above a threshold are eliminated or combined based on which option gives the best classification performance.*/
if $CMI(a, b|C) > \omega$ **then**
Eliminate or Combine a and b
end if
end for
end for

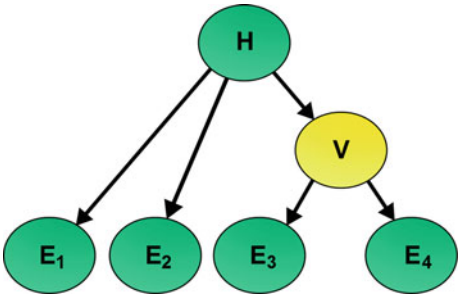


Fig. 4.6 An example of node creation for making two dependent attributes independent. Node V is inserted in the model given that E_3 and E_4 are not conditionally independent given H

References [9, 19] introduce an alternative operation for modifying the structure of a NBC, which consists in adding a new attribute that makes two dependent attributes independent; see Fig. 4.6. This new attribute is a kind of virtual or hidden node in the model, for which we do not have any data for learning its parameters. An alternative for estimating the parameters of hidden variables in Bayesian networks, such as in this case, is based on the Expectation-Maximization (EM) procedure, and it is described in Chap. 8.

All previous classifiers consider that there is a single class variable; that is, each instance belongs to one and only one class. Next we consider that an instance can belong to several classes, which is known as *multidimensional* classification.

4.6 Multidimensional Bayesian Classifiers

Several important problems need to predict several classes simultaneously. For example: text classification, where a document can be assigned to several topics; gene classification, as a gene may have different functions; image annotation, as an image may include several objects; among others. These are examples of *multi-dimensional classification*, in which more than one class can be assigned to an object. Formally, the *multi-dimensional classification* problem corresponds to searching for a function h that assigns to each instance represented by a vector of m features $\mathbf{X} = (X_1, \ldots, X_m)$ a vector of d class values $\mathbf{C} = (C_1, \ldots, C_d)$. The h function should assign to each instance $\mathbf{X}$ the most likely combination of classes, that is:

$$ArgMax_{c_1,\ldots,c_d} P(C_1 = c_1, \ldots, C_d = c_d|\mathbf{X}) \tag{4.17}$$

Multi-label classification is a particular case of multi-dimensional classification, where all class variables are binary. In the case of multi-label classification, there are two basic approaches: *binary relevance* and *label power-set* [21]. Binary relevance approaches transform the multi-label classification problem into d independent binary classification problems, one for each class variable, $C_1, \ldots, C_d$. A classifier is independently learned for each class and the results are combined to determine the